



WSQ COURSE SLIDES · CV0-004

WSQ CompTIA Certified Cloud+ Training

WSQ Course Code: TGS-2024049214 · 36 hands-on labs · 6 exam domains
Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Trainer: Dr. Alfred Ang · Version 1.0 · 1 Jul 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



COURSE ADMINISTRATION

Welcome & Housekeeping

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer/administrator displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your mobile phone camera and submit your attendance.
- A minimum of 75% attendance is required to be eligible for assessment and funding.

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

About the Trainer



Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte. Ltd.

ROLE

Principal Trainer & Cloud/DevOps Consultant, Tertiary Infotech Academy.

CERTIFICATION

CompTIA Cloud+ / cloud & DevOps certified practitioner (PhD).

DELIVERS

WSQ courses on cloud, containers, Kubernetes, IaC and DevSecOps.

EXPERIENCE

15+ years designing, deploying and teaching cloud solutions.

Let's Know Each Other

- Your name and organisation / role.
- Your current experience with cloud (AWS / Azure / GCP / on-prem).
- Do you use containers, CI/CD or Infrastructure as Code today?
- What you most want to take away from this course.

Ground Rules

1

Set your mobile phone to silent mode.

2

Participate actively — no question is too small.

3

Reset the Killercode playground between stateful labs.

4

One conversation at a time.

5

Be punctual; return from breaks on time.

6

75% attendance is required for funding.

LMS / TMS

- Access your course materials, attendance and assessment on the LMS/TMS portal.
- Portal: <https://lms-tms.tertiaryinfotech.com>
- Download the slides and Learner Guide for reference during the open-book assessment.

Lesson Plan — 2 Days (9:00 AM – 6:00 PM)

Day 1

- **Day 1 — Architecture, Deployment & Operations**

- Domain 1: Cloud Architecture (Labs 1–8)
- Domain 2: Deployment (Labs 9–15)
- Domain 3: Operations begins (Labs 16–18)
- Breaks: 10-min AM/PM · 45-min lunch

Day 2 & assessment

- **Day 2 — Security, DevOps & Troubleshooting**

- Domain 3: Operations (Labs 19–21)
- Domain 4: Security (Labs 22–28)
- Domain 5: DevOps (29–32) · Domain 6: Troubleshooting (33–36)
- Assessment 4:00–6:00 PM: 1 hr WA + 1 hr PP

Learning Outcomes

1

Domain 1 — Cloud Architecture

Cloud Architecture

2

Domain 2 — Deployment

Provisioning and configuring cloud resources

3

Domain 3 — Operations

Running cloud systems day-to-day

4

Domain 4 — Security

Securing the cloud

5

Domain 5 — DevOps Fundamentals

DevOps Fundamentals

6

Domain 6 — Troubleshooting

Troubleshooting

The Exam — CompTIA Cloud+ CV0-004

1

Exam code

CV0-004

2

Questions

Maximum of 90 · Multiple-choice and performance-based

3

Length

90 minutes

4

Passing score

750 (on a scale of 100–900)

5

Recommended experience

2–3 yrs sysadmin/cloud; Network+ & Server+

6

Delivery

100% hands-on on the free Killercodea Ubuntu Playground

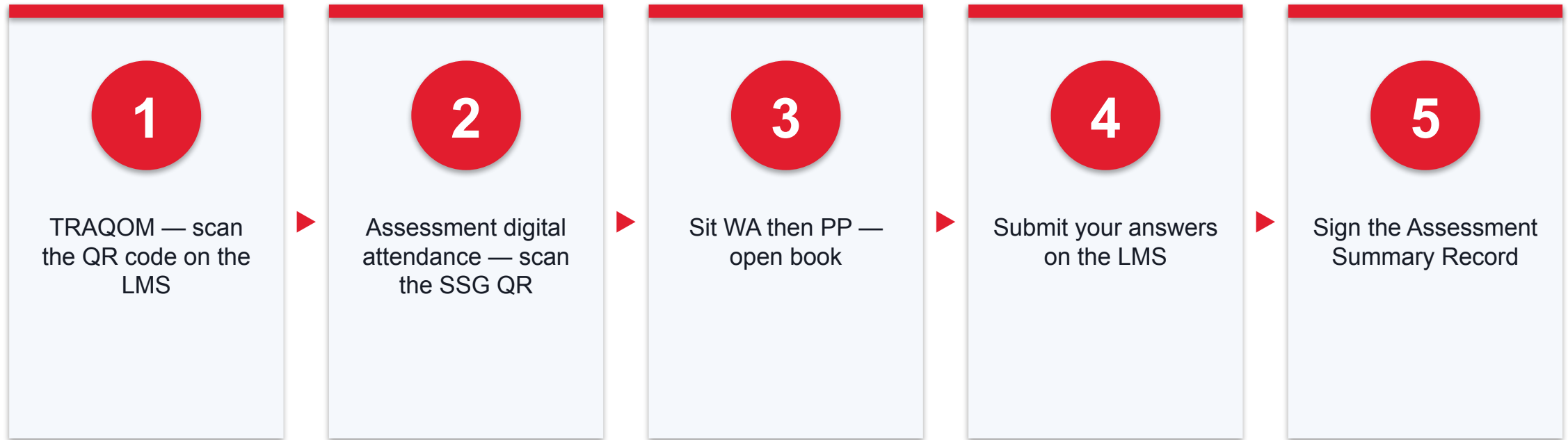
Briefing for Assessment

- Place phones and other materials under the table or on the floor.
- No photos or recording of assessment scripts.
- No discussion during the assessment.
- Use a black/blue pen for hard-copy assessments.
- No liquid paper / correction tape.
- Scripts are collected when time is up.

Assessment

- Written Assessment (WA) — 1 hour: open-ended short-answer knowledge questions.
- Practical Performance (PP) — 1 hour: hands-on tasks from the class labs.
- Format: Open book — slides, Learner Guide and approved materials only.
- You must be assessed Competent (C) in both WA and PP.
- Assessment runs 4:00–6:00 PM on Day 2.

Assessment Flow





CORE CONCEPTS

Cloud Foundations You'll Use All Course

Cloud Foundations

1

Shared Responsibility

The provider is never responsible for everything — you always own your data and access.

2

Service Models

IaaS, PaaS, SaaS and FaaS abstract away progressively more of the stack.

3

Deployment Models

Public, private, hybrid and community differ in tenancy, control and cost.

4

Elasticity & HA

Scale horizontally and remove single points of failure across availability zones.

5

Everything-as-Code

Infrastructure, configuration and pipelines are versioned, reviewable code.

6

Observability

Metrics, logs and traces together make a system observable and debuggable.

WHY IT MATTERS

The cloud is built, run and secured as code.

Every lab in this course is a hands-on reflection of a CV0-004 exam objective — build it, break it, fix it.

DOMAIN 1 · EXAM WEIGHT 23%

Cloud Architecture

Cloud concepts, service and deployment models, the shared responsibility model, high availability, networking (VPC), storage tiers, virtualization, databases and cloud-native / microservices design.

Key Concepts — Cloud Architecture

1

Cloud Service Models & Shared Responsibility

The four service models — IaaS, PaaS, SaaS, FaaS — and what each abstracts away

2

High Availability with HAProxy & Keepalived

Multi-AZ load balancing and round-robin distribution across backends

3

Cloud Networking with VPC Namespaces

A VPC/subnet is a kernel network feature (network namespace = subnet)

4

Storage Tiers (Block, Object, File)

Three storage types: block (EBS), object (S3), file (EFS/Azure Files)

5

Microservices & Service Discovery

Services register / deregister with a service registry (Consul)

6

Containerization with Docker

Image lifecycle: build, tag, run, push (Dockerfile FROM/COPY/EXPOSE)

7

Virtualization with QEMU/KVM

Hypervisor virtualization vs containerization (own kernel vs shared kernel)

8

Relational vs Non-Relational Databases

Relational model: strict schema, ACID, foreign keys, JOINS (Postgres)

Hands-On Labs — Cloud Architecture

Labs 1–3

- Lab 1: Cloud Service Models & Shared Responsibility
- Lab 2: High Availability with HAProxy & Keepalived
- Lab 3: Cloud Networking with VPC Namespaces

Labs 4–6

- Lab 4: Storage Tiers (Block, Object, File)
- Lab 5: Microservices & Service Discovery
- Lab 6: Containerization with Docker

Labs 7–8

- Lab 7: Virtualization with QEMU/KVM
- Lab 8: Relational vs Non-Relational Databases

Cloud Service Models & Shared Responsibility

GOAL Run a local stand-in for each cloud service model and articulate who is responsible for what under the shared responsibility model.

KEY CONCEPTS

- The four service models — IaaS, PaaS, SaaS, FaaS — and what each abstracts away
- Shared responsibility: who owns data, app, runtime, OS and hardware at each layer
- FaaS traits: event-driven, scales to zero, billed per invocation
- Mapping real services (EC2, managed Postgres, M365, Lambda) to each model

LAB STEPS

- 1 Install base tools
- 2 IaaS simulation (bare Ubuntu container)
- 3 PaaS simulation (managed Postgres)
- 4 SaaS simulation (Nextcloud)
- 5 FaaS simulation (Python handler)
- 6 Map the responsibility matrix
- 7 Cleanup

TOOLS

- Docker
- curl
- jq
- nginx
- postgres:16
- nextcloud

High Availability with HAProxy & Keepalived

GOAL Build a health-checked load balancer across two AZ backends and add VRRP failover so a virtual IP survives a load-balancer failure.

KEY CONCEPTS

- Multi-AZ load balancing and round-robin distribution across backends
- Active health checks that auto-remove unhealthy instances
- Keepalived / VRRP floating virtual IP (VIP) for LB redundancy
- Active-passive HA: the BACKUP node takes the VIP when the MASTER dies

LAB STEPS

- 1 Install HAProxy & Keepalived
- 2 Launch two backend 'AZ' web servers
- 3 Configure HAProxy with health checks
- 4 Verify load balancing
- 5 Simulate an AZ failure
- 6 Add Keepalived for VRRP failover
- 7 Cleanup

TOOLS

- HAProxy
- Keepalived
- Docker
- nginx:alpine
- curl

Cloud Networking with VPC Namespaces

GOAL Construct a full VPC — two subnets, a router, a security-group rule and a NAT gateway — entirely from Linux network namespaces.

KEY CONCEPTS

A VPC/subnet is a kernel network feature (network namespace = subnet)

Route tables, default routes and IP forwarding for inter-subnet routing

Security groups operate per-flow at L3/L4 (iptables = AWS SG / Azure NSG)

NAT gateway (MASQUERADE) lets private subnets reach the internet

LAB STEPS

- 1 Install tools
- 2 Create the VPC subnets (namespaces)
- 3 Wire the subnets to the router (veth pairs)
- 4 Add default routes (route table)
- 5 Add a security-group rule (deny ICMP B→A)
- 6 Add a NAT gateway
- 7 Cleanup

TOOLS

iproute2 (ip netns)

iptables

sysctl

ping

Storage Tiers (Block, Object, File)

GOAL Create one block, one object and one file store locally, then benchmark IOPS to explain why storage tiers (and their prices) differ.

KEY CONCEPTS

- Three storage types: block (EBS), object (S3), file (EFS/Azure Files)
- Object storage is key-addressed over HTTP with no filesystem; file is POSIX/NFS
- Measuring random-write IOPS with fio to compare tier performance
- Hot / warm / cold / archive tiering and how cost follows performance

LAB STEPS

- 1 Install tools
- 2 Create a BLOCK device (EBS-like)
- 3 Create an OBJECT store with MinIO (S3-like)
- 4 Create a FILE share with NFS
- 5 Compare IOPS between tiers with fio
- 6 Tier mapping table
- 7 Cleanup

TOOLS

- fio
- losetup/mkfs.ext4
- MinIO + mc
- NFS
- Docker

Microservices & Service Discovery

GOAL Deploy three loosely-coupled microservices behind Consul and demonstrate name-based service discovery, fan-out and resilience under a single failure.

KEY CONCEPTS

- Services register / deregister with a service registry (Consul)
- Clients resolve services by name (users.service.consul), not IP
- Fan-out pattern: one request triggers parallel calls to many services
- Loose coupling: killing one service leaves the others running

LAB STEPS

- 1 Install Docker & create a network
- 2 Run Consul (registry)
- 3 Deploy three microservices
- 4 Register them with Consul
- 5 Discover a service by name (DNS)
- 6 Fan-out call
- 7 Kill one service — others survive
- 8 Cleanup

TOOLS

- HashiCorp Consul
- Docker bridge network
- curl
- jq
- dig

Containerization with Docker

GOAL Practise every core containerization sub-objective — build/run/expose, ephemeral vs persistent storage, public/private registries and a Compose preview.

KEY CONCEPTS

- Image lifecycle: build, tag, run, push (Dockerfile FROM/COPY/EXPOSE)
- Port mapping (-p host:container)
- Ephemeral storage vs persistent volumes
- Public vs private image registries (Docker Hub vs a local registry:2)

LAB STEPS

- 1 Install Docker
- 2 Build a custom image
- 3 Stand-alone run with port mapping
- 4 Ephemeral storage
- 5 Persistent volume
- 6 Pull from a public registry
- 7 Run a private registry
- 8 Compose orchestration preview
- 9 Cleanup

TOOLS

- Docker
- Dockerfile
- registry:2
- Docker Compose

Virtualization with QEMU/KVM

GOAL Create and clone a QEMU VM to understand core virtualization concepts and contrast VMs with containers.

KEY CONCEPTS

- Hypervisor virtualization vs containerization (own kernel vs shared kernel)
- qcow2 copy-on-write disks and linked clones (basis of snapshots/templates)
- Hardware-assisted virtualization (Intel vmx / AMD svm)
- VM network types (user-mode NAT vs bridged) and host affinity / CPU pinning

LAB STEPS

- 1 Install QEMU & libvirt
- 2 Create a virtual disk (qcow2)
- 3 Clone the disk (linked clone)
- 4 Examine host virtualization capability
- 5 Boot a tiny VM
- 6 VM network types
- 7 Host affinity
- 8 VM vs Container summary
- 9 Cleanup

TOOLS

- QEMU
- qemu-img
- libvirt
- bridge-utils
- Alpine ISO

Relational vs Non-Relational Databases

GOAL Run and compare a self-managed relational DB against a document DB to learn when to choose each and how managed services shift responsibility.

KEY CONCEPTS

- Relational model: strict schema, ACID, foreign keys, JOINS (Postgres)
- Document model: schemaless, nested documents, built-in sharding (MongoDB)
- Self-managed vs provider-managed responsibility split
- Workload-to-engine selection and provider-managed equivalents (RDS, Cosmos...)

LAB STEPS

- 1 Install Docker
- 2 Self-managed Postgres
- 3 Create a relational schema
- 4 Provider-managed-style MongoDB
- 5 Compare strengths
- 6 Provider-managed equivalents
- 7 Cleanup

TOOLS

- Docker
- PostgreSQL 16 + psql
- MongoDB + mongosh

DOMAIN 2 · EXAM WEIGHT 19%

Deployment

Provisioning and configuring cloud resources: deployment models, zero-downtime release strategies (blue-green, canary, rolling), Infrastructure as Code, Configuration as Code and scripting logic.

Key Concepts — Deployment

1

Public, Private & Hybrid Cloud Models

Public (multi-tenant, pay-per-use) vs private (single-tenant, high CapEx)

2

Blue-Green Deployment with Nginx

Blue-green keeps two parallel environments; cutover is a config change

3

Canary Deployment with HAProxy

Canary = a small percentage of real traffic on the new version first

4

Rolling Deployment with Docker Compose

Rolling = replace replicas one (or a batch) at a time while others serve

5

Infrastructure as Code with Terraform

IaC: declarative configuration becomes real cloud resources

6

Configuration as Code with Ansible

Configuration as Code: YAML playbooks declare desired state

7

JSON & YAML Scripting Logic

Read, write, validate and convert JSON and YAML (same data, two formats)

Hands-On Labs — Deployment

Labs 9–11

- Lab 9: Public, Private & Hybrid Cloud Models
- Lab 10: Blue-Green Deployment with Nginx
- Lab 11: Canary Deployment with HAProxy

Labs 12–14

- Lab 12: Rolling Deployment with Docker Compose
- Lab 13: Infrastructure as Code with Terraform
- Lab 14: Configuration as Code with Ansible

Labs 15–15

- Lab 15: JSON & YAML Scripting Logic

Public, Private & Hybrid Cloud Models

GOAL Simulate public (LocalStack), private (MinIO) and hybrid deployment models to compare their cost, control and elasticity trade-offs.

KEY CONCEPTS

Public (multi-tenant, pay-per-use) vs private (single-tenant, high CapEx)

Hybrid: the same S3 API spans both clouds and enables cloud bursting

Community cloud is a policy/access concept, not new infrastructure

A consistent API (S3) can span deployment models

LAB STEPS

- 1 Install tools
- 2 Stand up a 'public' cloud (LocalStack)
- 3 Stand up a 'private' cloud (MinIO)
- 4 Hybrid: copy data between them
- 5 Community cloud concept
- 6 Decision matrix
- 7 Cleanup

TOOLS

LocalStack

MinIO + mc

AWS CLI

WireGuard

Docker

Blue-Green Deployment with Nginx

GOAL Run blue (v1) and green (v2) side by side and switch live traffic instantly via an Nginx reload to achieve zero-downtime deployment.

KEY CONCEPTS

Blue-green keeps two parallel environments; cutover is a config change

Atomic traffic cutover via 'nginx -s reload' with no dropped connections

Smoke-test the new version out of band before flipping traffic

Instant, symmetrical rollback; trade-off is 2× resources

LAB STEPS

- 1 Install Nginx & Docker
- 2 Run Blue (v1) and Green (v2)
- 3 Point Nginx at Blue
- 4 Smoke-test Green out of band
- 5 Switch traffic to Green (cutover)
- 6 Instant rollback
- 7 Cleanup

TOOLS

Nginx

Docker (nginx:alpine)

curl

Canary Deployment with HAProxy

GOAL Gradually shift live traffic from v1 to v2 using HAProxy server weights to safely validate a new release before full promotion.

KEY CONCEPTS

Canary = a small percentage of real traffic on the new version first

Weight-based routing (90/10 → 50/50 → 100%) = ALB weighted target groups / Istio

Gradual promotion and instant weight-reset rollback

Always pair canary with metrics and alerts

LAB STEPS

- 1 Install HAProxy & Docker
- 2 Run v1 (stable) and v2 (canary)
- 3 HAProxy at 90/10
- 4 Increase to 50/50
- 5 Promote v2 to 100%
- 6 Rollback (kill the canary)
- 7 Cleanup

TOOLS

HAProxy

Docker (nginx:alpine)

curl

Rolling Deployment with Docker Compose

GOAL Deploy three load-balanced replicas and upgrade them one at a time to perform a zero-downtime rolling update.

KEY CONCEPTS

Rolling = replace replicas one (or a batch) at a time while others serve

Produces a brief mixed-version window; cheaper than blue-green

Load balancing across replicas via HAProxy server-template + resolvers

Always pair with a readiness check before progressing

LAB STEPS

- 1 Install Docker Compose
- 2 Define the stack
- 3 Upgrade replicas one at a time
- 4 Compare strategies
- 5 Cleanup

TOOLS

Docker Compose

HAProxy

curl

Infrastructure as Code with Terraform

GOAL Write a Terraform config that provisions an S3 bucket on LocalStack and experience plan/apply, state, drift detection and versioning.

KEY CONCEPTS

- IaC: declarative configuration becomes real cloud resources
- The plan-before-apply workflow
- Drift detection compares real vs desired state
- State (terraform.tfstate) is the source of truth and must be stored safely

LAB STEPS

- 1 Install Terraform & LocalStack
- 2 Write the Terraform configuration
- 3 Init, plan, apply
- 4 Drift detection
- 5 Version your code (Git)
- 6 Apply a change (test → apply)
- 7 Cleanup

TOOLS

- Terraform
- LocalStack
- awslocal
- Git
- Docker

Configuration as Code with Ansible

GOAL Write an Ansible playbook that installs and configures Nginx idempotently and prove drift correction.

KEY CONCEPTS

Configuration as Code: YAML playbooks declare desired state

Idempotency — tasks only act when current state differs (changed vs ok)

Re-running restores desired state after manual tampering

Variables, conditionals (when:) and roles compose larger configs

LAB STEPS

- 1 Install Ansible
- 2 Define inventory (localhost)
- 3 Write a playbook (YAML)
- 4 First run (changes everything)
- 5 Idempotency test (run again)
- 6 Detect drift / repeatability
- 7 Variables & conditionals
- 8 Roles preview
- 9 Cleanup

TOOLS

Ansible

ansible-galaxy

YAML

curl

JSON & YAML Scripting Logic

GOAL Parse, transform, validate and convert JSON ↔ YAML while exercising the exam's scripting-logic primitives.

KEY CONCEPTS

Read, write, validate and convert JSON and YAML (same data, two formats)

Shared data types: string, number, boolean, array, object

Scripting-logic primitives: variables, conditionals, operators, functions (jq)

Validate at the boundary — fail early on malformed input

LAB STEPS

- 1 Install parsers
- 2 Write JSON
- 3 Same data in YAML
- 4 Convert between formats
- 5 Variables, conditionals, operators
- 6 Validate at the boundary
- 7 Web validators
- 8 Cleanup

TOOLS

jq

yq

diff

web validators

DOMAIN 3 · EXAM WEIGHT 17%

Operations

Running cloud systems day-to-day: observability (metrics, logs, traces), alerting, auto-scaling, backup & disaster recovery, and patching / lifecycle management.

Key Concepts — Operations

1

Observability with Prometheus & Grafana

Metrics pillar: Prometheus scrapes a source (node-exporter) on an interval

2

Centralized Logging with the ELK Stack

The centralized logging pipeline: collect → ship → store → query

3

Distributed Tracing with Jaeger

Tracing is the third pillar of observability (metrics/logs/traces)

4

Horizontal Auto-Scaling Simulation

Three scaling trigger families: triggered/load-based, scheduled, manual

5

Backup & Recovery with restic

Full vs incremental vs differential backup semantics and restore trade-offs

6

Patching & Lifecycle Management

The full lifecycle: provision → configure → monitor → patch → scale → backup → decommission

Hands-On Labs — Operations

Labs 16–17

- Lab 16: Observability with Prometheus & Grafana
- Lab 17: Centralized Logging with the ELK Stack

Labs 18–19

- Lab 18: Distributed Tracing with Jaeger
- Lab 19: Horizontal Auto-Scaling Simulation

Labs 20–21

- Lab 20: Backup & Recovery with restic
- Lab 21: Patching & Lifecycle Management

Observability with Prometheus & Grafana

GOAL Scrape a node's metrics with Prometheus, visualise them in Grafana and trigger a CPU alert.

KEY CONCEPTS

Metrics pillar: Prometheus scrapes a source (node-exporter) on an interval

Grafana visualises Prometheus as a data source

Threshold alerting: a PromQL rule fires when a condition holds for a duration

The operational loop: alerting → triage → response

LAB STEPS

- 1 Install Docker
- 2 Run node-exporter (metrics source)
- 3 Configure & run Prometheus
- 4 Run Grafana + add data source
- 5 Trigger the alert (stress-ng)
- 6 Three pillars summary
- 7 Cleanup

TOOLS

Prometheus

Grafana

node-exporter

stress-ng

Docker

Centralized Logging with the ELK Stack

GOAL Run ELK, ship logs into it, and query and retain them through the Elasticsearch API.

KEY CONCEPTS

The centralized logging pipeline: collect → ship → store → query

Elasticsearch (store/index), Logstash (ingest/transform), Kibana (query/visualise)

Retention enforced with ILM (Index Lifecycle Management) policies

ELK and OpenSearch share the same query DSL

LAB STEPS

- 1 Install Docker
- 2 Run Elasticsearch
- 3 Run Kibana
- 4 Run Logstash with a syslog input
- 5 Ship test logs
- 6 Retention policy (ILM)
- 7 Aggregation query
- 8 Cleanup

TOOLS

Elasticsearch

Logstash

Kibana

Docker

curl

jq

Distributed Tracing with Jaeger

GOAL Run Jaeger, instrument a Python 'checkout' path with OpenTelemetry and view a distributed trace of parent/child spans.

KEY CONCEPTS

Tracing is the third pillar of observability (metrics/logs/traces)

Spans are the unit; a trace is a set of causally-linked spans across services

OpenTelemetry is the vendor-neutral instrumentation API (OTLP export)

A trace reveals the critical path — the slowest span — of one request

LAB STEPS

- 1 Install Docker & Python deps
- 2 Run Jaeger
- 3 Instrument a 'checkout' path
- 4 Query traces via the Jaeger API
- 5 How tracing helps incident triage
- 6 Cleanup

TOOLS

Jaeger

OpenTelemetry Python SDK

Docker

python3

curl

Horizontal Auto-Scaling Simulation

GOAL Simulate triggered, scheduled and manual horizontal scaling using a Bash controller that watches CPU and adds/removes Docker replicas.

KEY CONCEPTS

- Three scaling trigger families: triggered/load-based, scheduled, manual
- An auto-scaler is a control loop that watches a metric and adjusts replicas
- Horizontal scaling (add replicas) vs vertical scaling (resize one instance)
- Horizontal scaling is the cloud-native default (AWS ASG, K8s HPA)

LAB STEPS

- 1 Install Docker Compose & tools
- 2 Define a scalable service
- 3 Triggered (load-based) scaling
- 4 Scheduled scaling
- 5 Manual scaling
- 6 Horizontal vs vertical
- 7 Cleanup

TOOLS

- Docker Compose
- stress-ng
- cron
- nginx:alpine

Backup & Recovery with restic

GOAL Perform encrypted full/incremental/differential backups to off-site S3 storage, then test recoverability and verify integrity.

KEY CONCEPTS

- Full vs incremental vs differential backup semantics and restore trade-offs
- Encrypted, deduplicating backups to off-site S3-compatible storage
- Recoverability testing (granular and bulk restore) is mandatory
- Integrity verification and retention policies (keep-daily/weekly + prune)

TOOLS

restic

MinIO

Docker

LAB STEPS

- 1 Install restic & MinIO
- 2 Configure restic for S3
- 3 Full backup
- 4 Incremental backup
- 5 Differential-style snapshot
- 6 Granular restore
- 7 Bulk restore
- 8 Integrity check
- 9 Retention policy
- 10 Backup matrix
- 11 Cleanup

Patching & Lifecycle Management

GOAL Simulate a cloud resource lifecycle from provision through minor patch, major upgrade, testing and clean decommissioning.

KEY CONCEPTS

- The full lifecycle: provision → configure → monitor → patch → scale → backup → decommission
- Minor patches (no behaviour change) vs major upgrades (breaking) risk profiles
- Persistent data must survive upgrades; ephemeral state may vanish
- EoL/EoS items must be replaced, not patched; decommission = archive + revoke + delete

LAB STEPS

- 1 Install patching tools
- 2 Provision phase
- 3 Apply minor patches
- 4 Major upgrade (test first)
- 5 Persistent vs ephemeral data
- 6 End of support / end of life
- 7 Decommissioning
- 8 Lifecycle summary

TOOLS

- apt
- unattended-upgrades
- Docker
- curl



DOMAIN 4 · EXAM WEIGHT 19%

Security

Securing the cloud: vulnerability management, CIS hardening, IAM & RBAC, secure access (bastion + MFA), secrets management, web application firewalls and runtime threat detection.

Key Concepts — Security

1

Vulnerability Scanning with Trivy

The four-step vulnerability-management workflow: scope → identify → assess → remediate

2

CIS Benchmark Audit with Lynis

Benchmarks (CIS) are objective, measurable targets; hardening closes the gap

3

IAM & RBAC with Keycloak

IAM building blocks: identity provider, realms (tenants), users, roles, clients

4

Bastion Host with SSH Key + MFA

SSH public-key authentication replacing passwords

5

Secrets Management with HashiCorp Vault

Never hard-code secrets; reference them from a secrets manager (Vault KV v2)

6

Web Application Firewall with ModSecurity

WAFs inspect HTTP semantics (query strings, bodies, headers) — not just IP/port

7

Detecting Suspicious Activity with Falco

Runtime security uses kernel signals (eBPF) to spot bad behaviour

Hands-On Labs — Security

Labs 22–24

- Lab 22: Vulnerability Scanning with Trivy
- Lab 23: CIS Benchmark Audit with Lynis
- Lab 24: IAM & RBAC with Keycloak

Labs 25–27

- Lab 25: Bastion Host with SSH Key + MFA
- Lab 26: Secrets Management with HashiCorp Vault
- Lab 27: Web Application Firewall with ModSecurity

Labs 28–28

- Lab 28: Detecting Suspicious Activity with Falco

Vulnerability Scanning with Trivy

GOAL Scan a container image, filesystem and IaC config with Trivy following the scope → identify → assess → remediate workflow.

KEY CONCEPTS

- The four-step vulnerability-management workflow: scope → identify → assess → remediate
- CVE (identifier) + CVSS (severity) as the universal vulnerability vocabulary
- Three scan surfaces: container image, filesystem (secrets), IaC config
- The CI 'gate' — a non-zero exit code fails the build

LAB STEPS

- 1 Install Trivy
- 2 Scoping
- 3 Identify (image scan)
- 4 Assess (CVSS)
- 5 Remediate (patched base image)
- 6 Filesystem scan
- 7 IaC scan
- 8 Daily automation / CI gate

TOOLS

Trivy

Docker

sample Terraform

CIS Benchmark Audit with Lynis

GOAL Audit the VM against the CIS Ubuntu Benchmark with Lynis, score it, remediate findings and re-audit to confirm improvement.

KEY CONCEPTS

- Benchmarks (CIS) are objective, measurable targets; hardening closes the gap
- The Lynis hardening index is a quantifiable score
- Findings map to specific CIS controls (file permissions, kernel modules)
- Different benchmarks per layer: OS, Docker Bench, kube-bench, Prowler

LAB STEPS

- 1 Install Lynis
- 2 Baseline audit
- 3 Read warnings/suggestions
- 4 Remediate file permissions
- 5 Remediate kernel hardening
- 6 Re-audit and compare
- 7 Vendor-specific benchmarks
- 8 Cleanup

TOOLS

- Lynis
- chmod
- sysctl
- Docker Bench Security

IAM & RBAC with Keycloak

GOAL Stand up Keycloak and build a full IAM stack — realm, roles, users, an OAuth2 client — then authenticate and inspect RBAC roles in the JWT.

KEY CONCEPTS

- IAM building blocks: identity provider, realms (tenants), users, roles, clients
- OAuth 2.0 + OIDC token flow (bearer JWT access token)
- RBAC — roles propagate via JWT realm_access.roles claims
- Authentication models: local, federation/SAML, OIDC, LDAP, MFA

LAB STEPS

- 1 Run Keycloak
- 2 Get an admin token
- 3 Create a realm (tenant)
- 4 Create roles (RBAC)
- 5 Create a user & assign a role
- 6 Create an OAuth2 client
- 7 Authenticate (token-based)
- 8 Authentication models map
- 9 Cleanup

TOOLS

- Keycloak
- Docker
- curl
- jq

Bastion Host with SSH Key + MFA

GOAL Configure key-based SSH with TOTP MFA and route access through a locked-down bastion jump host — the canonical secure cloud-VM access pattern.

KEY CONCEPTS

- SSH public-key authentication replacing passwords
- TOTP multi-factor auth chained via AuthenticationMethods / PAM
- Bastion / jump-host pattern (internet → bastion → private subnet)
- Restricting bastion access to corporate CIDRs; audit trails for compliance

LAB STEPS

- 1 Install OpenSSH & Google Authenticator
- 2 Generate an SSH key
- 3 Disable password auth
- 4 Set up TOTP MFA
- 5 Create a bastion architecture
- 6 Restrict who can reach the bastion
- 7 Audit trail
- 8 Cleanup

TOOLS

- OpenSSH
- ssh-keygen
- Google Authenticator (PAM)
- UFW
- journalctl

Secrets Management with HashiCorp Vault

GOAL Run Vault to store static secrets, mint auto-expiring dynamic DB credentials and audit every access.

KEY CONCEPTS

- Never hard-code secrets; reference them from a secrets manager (Vault KV v2)
- Secret versioning for audit and rollback
- Dynamic, short-lived DB credentials with lease/TTL shrink the blast radius
- Audit logging of every secret access; cloud equivalents (Secrets Manager, Key Vault)

LAB STEPS

- 1 Run Vault in dev mode
- 2 Write/read a static secret (KV v2)
- 3 Versioned secrets
- 4 Dynamic database credentials
- 5 Audit log
- 6 Cloud equivalents
- 7 Cleanup

TOOLS

- HashiCorp Vault
- Docker
- PostgreSQL
- curl
- jq

Web Application Firewall with ModSecurity

GOAL Deploy Nginx + ModSecurity + the OWASP Core Rule Set and watch it block SQLi, XSS and path-traversal, then tune false positives.

KEY CONCEPTS

- WAFs inspect HTTP semantics (query strings, bodies, headers) — not just IP/port
- OWASP CRS catches the OWASP Top 10 out of the box (SQLi, XSS, LFI)
- Anomaly scoring with a block threshold and paranoia levels
- False-positive tuning done test → staging → prod; ACL vs WAF vs Security Group

LAB STEPS

- 1 Run a prebuilt Nginx+ModSecurity image
- 2 Benign request (allowed)
- 3 Trigger a SQL-injection rule
- 4 Trigger an XSS rule
- 5 Trigger a path-traversal rule
- 6 Inspect the WAF audit log
- 7 Tune (false-positive triage)
- 8 ACL vs WAF vs Security Group
- 9 Cleanup

TOOLS

- Docker (owasp/modsecurity-crs)
- Nginx
- OWASP CRS
- curl

Detecting Suspicious Activity with Falco

GOAL Run Falco and trigger runtime detections that map to CV0-004 attack categories, then route alerts to Prometheus / SIEM.

KEY CONCEPTS

- Runtime security uses kernel signals (eBPF) to spot bad behaviour
- Common detections: shell-in-container, sensitive-file reads, odd outbound connections
- Detections map to CV0-004 attack categories (privesc, cryptojacking, metadata abuse)
- Baseline-deviation alerting; events flow into Prometheus / SIEM

LAB STEPS

- 1 Install Falco
- 2 Tail Falco events
- 3 Trigger shell-in-container
- 4 Trigger a sensitive-file read
- 5 Trigger an unexpected outbound connection
- 6 Map to exam attack types
- 7 Baseline deviation alerting
- 8 Cleanup

TOOLS

- Falco (eBPF)
- Docker
- falcosidekick
- journalctl

DOMAIN 5 · EXAM WEIGHT 10%

DevOps Fundamentals

Source control and branching, CI/CD pipelines, web-service APIs (REST / GraphQL) and container orchestration with Kubernetes.

Key Concepts — DevOps Fundamentals

1

Git Source Control & Branching

Core Git workflow: commit → push → branch → PR → merge

2

CI/CD Pipeline with GitHub Actions (act)

A pipeline is YAML + jobs + dependencies (needs:) forming a DAG

3

REST & GraphQL APIs

REST is stateless, uses HTTP verbs, one resource per URL, described by OpenAPI

4

Container Orchestration with Kubernetes (k3s)

k3s is a minimal single-binary Kubernetes cluster installable in one command

Hands-On Labs — DevOps Fundamentals

Labs 29–30

- Lab 29: Git Source Control & Branching
- Lab 30: CI/CD Pipeline with GitHub Actions (act)

Labs 31–32

- Lab 31: REST & GraphQL APIs
- Lab 32: Container Orchestration with Kubernetes (k3s)

—

- —

Git Source Control & Branching

GOAL Exercise every CV0-004 source-control verb — commit, push, branch, merge, PR review, conflict resolution — against a self-hosted Gitea server.

KEY CONCEPTS

- Core Git workflow: commit → push → branch → PR → merge
- Feature branches and pull requests (the code-review sub-objective)
- Merge-conflict resolution as a normal part of the workflow
- Branch protection: require PR + approval, require CI green, block force-push

LAB STEPS

- 1 Install git & run Gitea
- 2 Create a repository
- 3 Clone, commit, push
- 4 Branch management
- 5 Open a Pull Request
- 6 Merge
- 7 Resolve a merge conflict
- 8 Branch protection
- 9 Cleanup

TOOLS

- git
- Gitea (self-hosted)
- curl
- jq

CI/CD Pipeline with GitHub Actions (act)

GOAL Build and run a local test → build → scan → deploy CI/CD pipeline with act, mapping each stage to CV0-004 CI/CD concepts.

KEY CONCEPTS

- A pipeline is YAML + jobs + dependencies (needs:) forming a DAG
- The canonical flow: test → build → scan → deploy
- Artifacts and packaging (container image + Dockerfile)
- Running Actions locally with act to iterate without pushing

LAB STEPS

- 1 Install Docker, git & act
- 2 Sample repo with a Python app
- 3 Define the workflow (ci.yml)
- 4 Run the workflow locally
- 5 Map to CV0-004 CI/CD concepts
- 6 Public vs private repositories
- 7 Other free CI options
- 8 Cleanup

TOOLS

- act (nektos/act)
- Docker
- git
- pytest
- Trivy

REST & GraphQL APIs

GOAL Call a public REST API, host a local GraphQL server, stream over WebSockets and preview pub/sub event-driven messaging.

KEY CONCEPTS

- REST is stateless, uses HTTP verbs, one resource per URL, described by OpenAPI
- GraphQL: a single endpoint where the client picks exactly which fields to return
- Comparison of styles: REST vs SOAP vs GraphQL vs gRPC
- WebSockets for real-time streams; pub/sub as the base of event-driven architecture

LAB STEPS

- 1 Install tools
- 2 Call a REST endpoint (CRUD)
- 3 Host a local GraphQL endpoint
- 4 Compare REST vs GraphQL vs SOAP vs RPC
- 5 WebSocket stream
- 6 Event-driven preview (Redis pub/sub)
- 7 Cleanup

TOOLS

- curl
- jq
- GraphQL Yoga
- websocat
- redis-cli

Container Orchestration with Kubernetes (k3s)

GOAL Install k3s and deploy, expose, scale, roll out/rollback, add persistence, self-healing and RBAC on a real Kubernetes cluster.

KEY CONCEPTS

- k3s is a minimal single-binary Kubernetes cluster installable in one command
- Deployments manage replica state; Services (NodePort) expose pods
- Horizontal scaling, rolling updates, rollout history and rollback
- Self-healing via liveness probes; PVCs for persistence; RBAC for authorization

LAB STEPS

- 1 Install k3s
- 2 Deploy an application
- 3 Scale (horizontal)
- 4 Rolling update
- 5 Persistent volume
- 6 Liveness probe (self-healing)
- 7 RBAC
- 8 Map K8s to exam objectives
- 9 Cleanup

TOOLS

- k3s
- kubectl
- YAML manifests
- curl

DOMAIN 6 · EXAM WEIGHT 12%

Troubleshooting

Methodically diagnosing and fixing deployment, network, security and container / resource issues using a repeatable triage workflow.

Key Concepts — Troubleshooting

1

Troubleshoot Deployment Issues

Resource allocation / sizing failures cause OOM kills — size to peak load

2

Troubleshoot Cloud Network Issues

DNS and NTP break 'everything else' when they fail

3

Troubleshoot TLS, Cipher & Auth Issues

Disable deprecated TLS (SSLv3/TLS 1.0/1.1); require TLS 1.2+ with AEAD ciphers

4

Troubleshoot Container & Resource Limits

CrashLoopBackOff = process exits immediately; fix the entrypoint so it stays up

Hands-On Labs — Troubleshooting

Labs 33–34

- Lab 33: Troubleshoot Deployment Issues
- Lab 34: Troubleshoot Cloud Network Issues

Labs 35–36

- Lab 35: Troubleshoot TLS, Cipher & Auth Issues
- Lab 36: Troubleshoot Container & Resource Limits

—

- —

Troubleshoot Deployment Issues

GOAL Reproduce and fix each CV0-004 6.1 deployment failure mode locally — resources, permissions, oversubscription, sizing, deprecation, quotas, regions.

KEY CONCEPTS

- Resource allocation / sizing failures cause OOM kills — size to peak load
- Permission misconfiguration fixed by mounting writable tmpfs/volumes
- Outdated/EoL definitions and deprecation are deployment risks — pin supported tags
- API throttling returns HTTP 429 — fix with quota increase, backoff, batching

TOOLS

- Docker
- curl
- jq
- region matrices

LAB STEPS

- 1 Setup
- 2 Resource allocation / sizing failure
- 3 Permission misconfiguration
- 4 Oversubscription
- 5 Outdated component definition
- 6 Deprecated functionality
- 7 Outage simulation
- 8 API throttling / service quota
- 9 Regional service availability
- 10 Decision tree

Troubleshoot Cloud Network Issues

GOAL Reproduce and resolve CV0-004 6.2 network failures using an L2→L7 diagnostic ladder (DHCP, DNS, NTP, NAT, HTTP codes, routing, IP overlap).

KEY CONCEPTS

- DNS and NTP break 'everything else' when they fail
- NAT (MASQUERADE + ip_forward) enables outbound from private subnets
- HTTP status codes localise the failure layer (4xx client, 5xx server, 429 rate-limit)
- IP overlap needs NAT/re-IP; DHCP scope exhaustion needs a bigger subnet

LAB STEPS

- 1 Install diagnostics
- 2 DNS failure → fix
- 3 NTP / time skew
- 4 NAT failure
- 5 HTTP status-code triage
- 6 Latency / bandwidth
- 7 IP overlap & scope exhaustion
- 8 Routing issues
- 9 VLAN / switching
- 10 Decision tree

TOOLS

- dig/nslookup
- ip
- iptables
- mtr
- chrony
- tcpdump

Troubleshoot TLS, Cipher & Auth Issues

GOAL Reproduce and fix CV0-004 6.3 security failures from the CLI — deprecated ciphers, privesc, unauthorized access, leaked credentials, vulnerabilities.

KEY CONCEPTS

Disable deprecated TLS (SSLv3/TLS 1.0/1.1); require TLS 1.2+ with AEAD ciphers

Detect privilege escalation via auditd file watches and SUID hunting

Detect brute force in SSH logs (fail2ban); find leaked credentials with gitleaks

TLS handshake errors map to specific root causes (missing CA, cipher mismatch, expiry)

LAB STEPS

- 1 Install tools
- 2 Deprecated cipher / TLS version
- 3 Test a public site's ciphers
- 4 Privilege-escalation detection
- 5 Unauthorized access / brute force
- 6 Leaked credentials in source control
- 7 Software vulnerability & unauthorized software
- 8 TLS handshake cheatsheet
- 9 Cleanup

TOOLS

openssl

nmap

auditd

fail2ban

gitleaks

Trivy

Troubleshoot Container & Resource Limits

GOAL Combine the Lab 33–35 methods to triage real container failures with a repeatable state → logs → stats → mounts → network workflow.

KEY CONCEPTS

- CrashLoopBackOff = process exits immediately; fix the entrypoint so it stays up
- OOMKilled is the smoking gun (OOMKilled=true); raise the limit or fix the leak
- Image-pull failures come from wrong names or missing registry auth
- The daily five: OOM, CrashLoop, ImagePull, DNS, read-only-fs; triage in a fixed order

LAB STEPS

- 1 Setup
- 2 CrashLoopBackOff
- 3 OOMKilled
- 4 Image pull failure
- 5 DNS inside containers
- 6 Read-only filesystem errors
- 7 Resource quota exhaustion
- 8 End-to-end triage script
- 9 Master decision flow
- 10 Closing summary

TOOLS

- docker (inspect/stats/logs)
- a triage.sh script
- k9s/ctop



WRAP-UP

Course Summary & Next Steps

What You Achieved

1

Domain 1 — Cloud Architecture

8 hands-on labs · 23% of the exam

2

Domain 2 — Deployment

7 hands-on labs · 19% of the exam

3

Domain 3 — Operations

6 hands-on labs · 17% of the exam

4

Domain 4 — Security

7 hands-on labs · 19% of the exam

5

Domain 5 — DevOps Fundamentals

4 hands-on labs · 10% of the exam

6

Domain 6 — Troubleshooting

4 hands-on labs · 12% of the exam

Courseware & Assessment on the LMS

- Download the slides and Learner Guide from the LMS for the open-book assessment.
- Complete the Written Assessment and Practical Performance on assessment day.
- Submit your answers on the LMS at <https://lms-tms.tertiaryinfotech.com/>.
- Re-run any lab from the Killercoda playground to revise before the CV0-004 exam.

Preparing for the CV0-004 Exam

- Redo every lab until the commands are automatic; review each lab's 'What you learned'.
- The exam has up to 90 multiple-choice and performance-based questions in 90 minutes.
- Passing score is 750 (scale 100–900).
- Book the exam through Pearson VUE when you are consistently scoring well on practice tests.

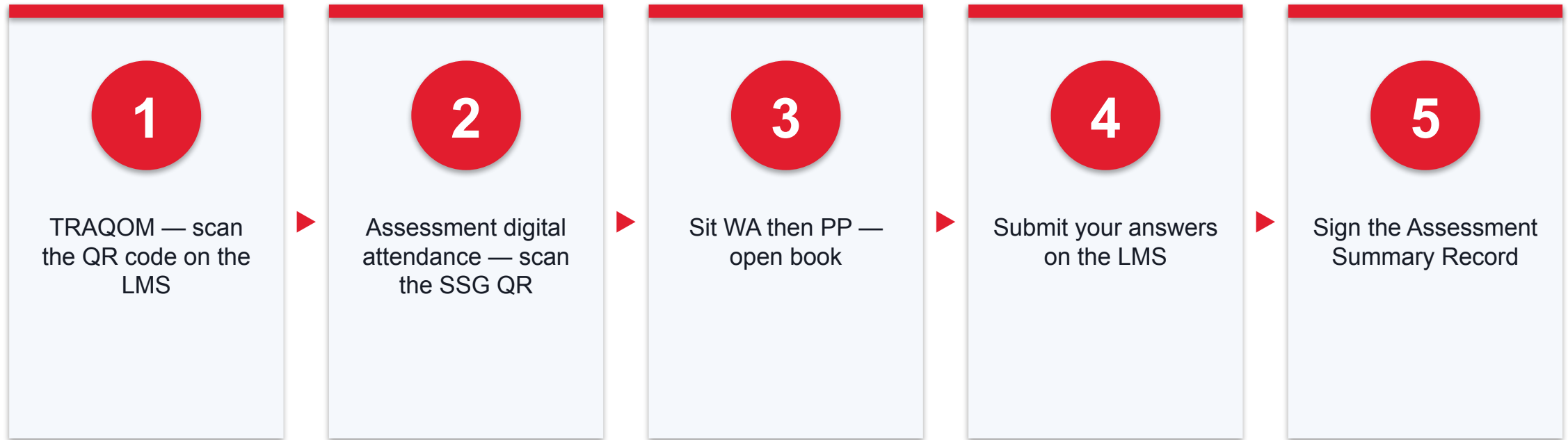
Practice Exam

- Sharpen your exam readiness with the Tertiary Infotech CompTIA Cloud+ practice exam.
- Practice exam: <https://exams.tertiaryinfotech.com/practice-exams/comptia/comptia-cloud-plus>
- Attempt it under timed conditions and review every explanation.
- Revisit any lab whose topic you miss, then re-take the practice exam.

Assessment

- Written Assessment (WA) — 1 hour. Practical Performance (PP) — 1 hour.
- Open book: slides, Learner Guide and approved materials only.
- Remember to take the Assessment digital attendance (TRAQOM · SSG).
- Submit your completed answers on the LMS at <https://lms-tms.tertiaryinfotech.com/>.

Assessment Flow



Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer/administrator displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your mobile phone camera and submit your attendance.
- A minimum of 75% attendance is required to be eligible for assessment and funding.

ALL THE BEST

Thank You!

You are now ready to build, run, secure and troubleshoot the cloud — and to sit the CompTIA Cloud+ CV0-004 exam.